# FRA143: Final Project - Robot Simulation

ให้นักศึกษาพัฒนาโปรแกรมจำลองการทำงานของหุ่นยนต์โดยใช้การออกแบบเชิงวัตถุ (OOP) และมีแสดงผลในรูปแบบตาราง (grid) โดยมีระยะเวลาทั้งหมด 4 สัปดาห์และสามารถทำงานเป็นกลุ่มได้ไม่เกินกลุ่มละ 4 คน

## Overview

ให้แต่ละกลุ่มออกแบบสถานการณ์ที่มีหุ่นยนต์หลายประเภท/หลายตัวทำงานอยู่ร่วมกันอยู่ในระบบ และพัฒนาโปรแกรมจำลองการทำงานของหุ่นยนต์ที่แสดงให้เห็นถึงสถานะของหุ่นยนต์ (ตำแหน่ง, state, etc) ตั้งแต่เริ่มต้นจนถึงเวลาสิ้นสุดโปรแกรม simulation (สามารถกำหนดเวลาให้เป็น discrete time step ได้)

เป้าหมายสำคัญของโปรเจกต์คือการออกแบบและประยุกต์ใช้คลาส ไม่จำเป็นต้องเน้นความซับซ้อนของระบบการทำงานหรือการความสมจริงทาง physics

- **ระยะเวลา:** 4 สัปดาห์
- **ขนาดกลุ่ม:** 1–4 คน
- **ภาษา:** C++ หรือ Python
- **การแสดงผล:** ต้องเป็นตาราง (grid) สามารถใช้ library เสริมได้ เช่น pygame, Qt, matplotlib หรือวาดด้วยตัวอักษรลงบน terminal ตรงๆ

## Scenario Setup

นักศึกษาสามารถเลือกสถานการณ์จากตัวอย่างข้างล่างนี้ หรือเสนอสถานการณ์ของกลุ่มเองมาใน proposal

| Scenario | Robot(s) | Tasks |
|---|---|---|
| **โรงงาน (Factory)** | หุ่นยนต์ขนของ, แขนกลเชื่อม, หุ่นยนต์ตรวจสอบ | ขนชิ้นงาน, เชื่อมชิ้นงาน, ตรวจคุณภาพ |
| **คลังสินค้า (Warehouse)** | หุ่นยนต์หยิบของ, หุ่นยนต์แพ็กของ | หยิบสินค้า, แพ็กกล่อง |
| **ร้านอาหาร (Restaurant)** | หุ่นยนต์ทำอาหาร, หุ่นยนต์เสิร์ฟ | ทำอาหาร, เสิร์ฟ, เก็บโต๊ะ |
| **โรงพยาบาล (Hospital)** | หุ่นยนต์ส่งยา, หุ่นยนต์ทำความสะอาด | ส่งยา, ส่งอาหาร, ทำความสะอาดห้อง |
| **ฟาร์ม (Farm)** | หุ่นยนต์ปลูก, หุ่นยนต์รดน้ำ, หุ่นยนต์เก็บเกี่ยว | ปลูก, รดน้ำ, เก็บเกี่ยว |

**Example: โรงงานที่มีหุ่นยนต์ 3 ประเภท ประเภทละ 1 ตัว**

โรงงานเป็นตารางขนาด 10 × 5 ช่อง มีคลังชิ้นส่วน (S) จุดตรวจสอบ (I) และแท่นชาร์จ (C) หุ่นยนต์มี 3 ชนิด:

- **หุ่นยนต์ขนของ (Carrier)** ทำงาน "ขนชิ้นงาน" ใช้ 4 หน่วยเวลาต่องาน
- **หุ่นยนต์เชื่อม (Welder)** ทำงาน "เชื่อมชิ้นงาน" ใช้ 3 หน่วยเวลาต่องาน
- **หุ่นยนต์ตรวจสอบ (Inspector)** ทำงาน "ตรวจคุณภาพ" ใช้ 2 ขั้นเวลาต่องาน

งานแต่ละงานอยู่ที่ช่องใดช่องหนึ่งบนตาราง (*) ในแต่ละขั้นเวลา หุ่นยนต์ที่ว่างจะรับงานที่ตัวเองทำได้ แล้วเดินไปยังช่องของงานทีละ 1 ช่อง เมื่อถึงแล้วจึงเริ่มทำงาน และเมื่องานเสร็จก็กลับไปว่าง ถ้าแสดงผลบน terminal แต่ละขั้นเวลาจะได้ผลประมาณนี้:

```
=== Step 5 ===
   0 1 2 3 4 5 6 7 8 9
0  S . . . . . . . . .
1  . . 1 . . . * . . .
2  . . . . . . . 2 . .
3  . * . . . . . . . .
4  C . . . . . . . 3 I

Robot  Type        Pos    Status   Task             Progress  Battery
R1     Carrier     (2,1)  MOVING   Move part #3     -         80%
R2     Welder      (7,2)  WORKING  Weld part #2     1/3       95%
R3     Inspector   (8,4)  MOVING   Inspect part #1  -         90%
Tasks: waiting 2 | in progress 3 | done 4
```

เมื่องานทั้งหมดเสร็จ โปรแกรมควรแสดงสรุปการทำงานทั้งหมด เช่น จำนวนขั้นเวลาที่ใช้ และจำนวนงานที่หุ่นยนต์แต่ละตัวทำ

## Requirements:

สิ่งที่โปรแกรมต้องมี:

1. มีโลกเป็นตาราง 2 มิติ ขนาดอย่างน้อย 10 × 5 ช่อง และแสดงผลเป็นตารางบนหน้าจอ
2. สร้างหุ่นยนต์อย่างน้อย 3 ตัว(ประเภทซ้ำกันได้) และรายการงานอย่างน้อย 5 งาน
3. ให้หุ่นยนต์แต่ละตัวมีชื่อ ชนิด ตำแหน่งบนตาราง สถานะ (เช่น ว่าง, กำลังเดิน, กำลังทำงาน) และค่าอย่างน้อย 1 ค่า เช่น แบตเตอรี่
4. ให้งานแต่ละงานมีชื่อ ชนิด ตำแหน่งบนตาราง และระยะเวลาเป็นจำนวนขั้นเวลา โดยหุ่นยนต์ทำได้เฉพาะงานที่ตรงกับชนิดของตัวเอง
5. จำลองการทำงานทีละ 1 timestep (หน่วยเวลา): มอบงานให้หุ่นยนต์ที่ว่าง, ให้หุ่นยนต์เดินไปยังงานทีละ 1 ช่อง, ทำงานเมื่อถึงที่หมาย, ปิดงานที่เสร็จ
6. แสดงตารางพร้อมตำแหน่งหุ่นยนต์ และแสดงสถานะของหุ่นยนต์และงาน ทุกๆ timestep
7. แสดงสรุปผลเมื่องานทั้งหมดเสร็จ

หมายเหตุ:

- ไม่จำเป็นต้องวางเส้นทางการเคลื่อนที่ที่ซับซ้อน สามารถกำหนดกฎพื้นฐานในการเคลื่อนที่ได้เช่นหุ่นยนต์เดินตามแนวนอนก่อนแล้วจึงเดินตามแนวตั้ง และหุ่นยนต์อยู่ช่องเดียวกันได้ (แต่ถ้ากลุ่มไหนต้องการทำระบบที่ซับซ้อนกว่านั้นก็ได้ เช่น เลือกเส้นทางที่สั้นที่สุดและไม่ชนกับหุ่นยนต์ตัวอื่นหรือมีสิ่งกีดขวางอื่นๆเพิ่มเข้ามาในระบบ)
- ถ้าใช้ terminal สามารถทำการล้างหน้าจอทุก timestep เพื่อให้ดูเหมือนภาพเคลื่อนไหวได้

### ข้อกำหนดด้าน OOP

เช่นเดียวกับโปรเจกต์เกมที่ผ่านมา โค้ดต้องแสดงแนวคิดทุกข้อด้านล่าง และในเอกสารออกแบบต้องระบุว่าแต่ละแนวคิดอยู่ตรงไหนในโค้ด

| Concept | สิ่งที่ต้องแสดง | ตัวอย่างในโรงงาน |
|---|---|---|
| **Encapsulation** | ข้อมูลเป็น private และเปลี่ยนได้ผ่าน methods เท่านั้น | แบตเตอรี่ลดได้ผ่านเมธอดของหุ่นยนต์ และต่ำกว่าศูนย์ไม่ได้ |
| **Inheritance** | มีคลาสลูกอย่างน้อย 2 คลาสจากคลาสแม่เดียวกัน | `Carrier`, `Welder`, `Inspector` สืบทอดจาก `Robot` |
| **Polymorphism** | เรียก method ผ่านคลาสแม่ แล้วแต่ละคลาสลูกทำงานต่างกัน โดยไม่ใช้ `if type == ...` | วนลูปเรียก `work()` กับหุ่นยนต์ทุกตัวในรายการเดียว |
| **Composition** | วัตถุหนึ่งมีวัตถุอื่นเป็นส่วนประกอบ | `Factory` มีรายการ `Robot` และรายการ `Task` |

| Concept | สิ่งที่ต้องแสดง | ตัวอย่างในโรงงาน |
|---|---|---|
| **Constructor และ destructor** | สร้างและคืนทรัพยากรของวัตถุอย่างถูกต้อง | สร้างหุ่นยนต์พร้อมชื่อและแบตเตอรี่เริ่มต้น |

### ขอบเขตตามขนาดกลุ่ม

กลุ่มที่มีสมาชิกมากขึ้นต้องมีชนิดหุ่นยนต์ ชนิดงาน และฟีเจอร์เสริมมากขึ้น

| สมาชิก | ชนิดหุ่นยนต์ | ชนิดงาน | Extra Feature(s) |
|---|---|---|---|
| 1-2 | 2 | 2 | 1 |
| 3 | 3 | 3 | 2 |
| 4 | 3 | 3 | 3 |

Example of Extra Features:

- หุ่นยนต์ต้องเดินไปชาร์จที่แท่นชาร์จเมื่อแบตเตอรี่ต่ำ
- หุ่นยนต์หลบกัน ไม่อยู่ช่องเดียวกัน
- มีสิ่งกีดขวางบนตารางที่หุ่นยนต์ต้องเดินอ้อม
- งานมีลำดับความสำคัญ (priority)
- หุ่นยนต์เสียแบบสุ่มและต้องรอซ่อม
- โหลดตาราง หุ่นยนต์ และงาน จากไฟล์ข้อความ

## ข้อกำหนดด้านการพัฒนาซอฟต์แวร์

- **Repository:** กลุ่มละ 1 repository บน GitHub และเชิญอาจารย์กับ TA เข้าร่วม สมาชิกทุกคนต้อง commit งานของตัวเอง
- **Pull request:** หลังสัปดาห์ที่ 1 ให้เพิ่มงานผ่าน pull request และให้สมาชิกอีกคนรีวิวก่อน merge (กลุ่ม 1 คนไม่ต้องรีวิว)
- **Tests:** มี unit test อย่างน้อยสำหรับคลาสหุ่นยนต์และคลาสงาน
- **README:** วิธี build และรันโปรแกรม และคำอธิบายสั้น ๆ ว่าโปรแกรมจำลองอะไร

## สิ่งที่ต้องส่งและ deadline

| Date | Task | Deliverable (สิ่งที่ต้องส่ง) |
|---|---|---|
| 11/11/2569 | เลือกสถานการณ์; ออกแบบ class diagram; ตั้งค่า repository | Project Proposal 1 หน้า พร้อม class diagram และลิงก์ repository |
| 23/11/2569 | คลาสหุ่นยนต์และคลาสงานทำงานได้; แสดงสถานะบน terminal, การจำลองครบตามข้อกำหนด และฟีเจอร์เสริมตามขนาดกลุ่ม; unit test | โค้ดใน repository |
| Final Exam Week | Change request; Presentation | Final codes, and demo |

### Final Exam Week (to be decided): Change request and final presentation

ก่อนการนำเสนอ 1 วันแต่ละกลุ่มจะได้รับข้อกำหนดใหม่หนึ่งข้อ และต้องทำให้เสร็จภายในเวลาที่กำหนด ตัวอย่างเช่น:

- เพิ่มหุ่นยนต์ชนิดใหม่
- เพิ่มงานชนิดใหม่
- หุ่นยนต์ชนิดหนึ่งทำงานเร็วขึ้นเมื่อแบตเตอรี่เต็ม

ถ้าออกแบบคลาสดี การเพิ่มสิ่งเหล่านี้ควรทำได้โดยเพิ่มคลาสใหม่ และแก้โค้ดเดิมเพียงเล็กน้อย

การนำเสนอใช้เวลาประมาณ 10 นาทีต่อกลุ่ม:

1. รันโปรแกรมให้ดู
2. อธิบาย class diagram และชี้ว่าแนวคิด OOP แต่ละข้ออยู่ตรงไหน
3. สาธิต change request
4. สมาชิกทุกคนตอบคำถามเกี่ยวกับโค้ด

### เกณฑ์การให้คะแนน

| เกณฑ์ | น้ำหนัก | สิ่งที่พิจารณา |
|---|---|---|
| การออกแบบ OOP | 35% | แสดงแนวคิดครบทุกข้อ; แต่ละคลาสมีหน้าที่ชัดเจน |
| การทำงานของโปรแกรม | 25% | ทำงานได้ครบตามข้อกำหนดและขอบเขตของกลุ่ม |
| Change request | 15% | เสร็จทันเวลา และแก้โค้ดเดิมน้อย |
| กระบวนการพัฒนา | 15% | commit จากสมาชิกทุกคน, pull request, unit test, README |
| การนำเสนอ | 10% | อธิบายได้ชัดเจน ตอบคำถามได้ |

คะแนนรายบุคคลอาจปรับตาม CW/code commits และการตอบคำถามในการนำเสนอ

### กติกา

- **เครื่องมือ AI** ใช้ได้ แต่สมาชิกทุกคนต้องอธิบายโค้ดทุกบรรทัดใน repository ได้ โค้ดที่อธิบายไม่ได้จะไม่ได้คะแนน
- **โค้ดจากภายนอก** (library, บทเรียนออนไลน์, หรือกลุ่มอื่น) ต้องระบุที่มาใน README
- **การมีส่วนร่วม:** สมาชิกแต่ละคนต้องควรมี commit ในส่วนที่ตนเองรับผิดชอบ สมาชิกที่ไม่มีผลงานปรากฏอาจได้คะแนนรายบุคคลลดลง